MODULE 3 – C# FUNDAMENTALS

Revision Notes

1. Introduction to C#

C# (C-Sharp) is a modern, object-oriented, type-safe programming language developed by Microsoft.

It is mainly used for:

- Web application development
- Desktop application development
- Game development
- Mobile application development
- Web API development

Features of C#:

- Simple and easy to learn
- Object-oriented
- Type-safe
- Platform-independent with .NET
- Automatic memory management
- Supports exception handling
- Supports asynchronous programming

2. .NET Framework and .NET

.NET is a development platform used to build and run applications.

Important components:

1. CLR (Common Language Runtime): Executes .NET applications and manages memory.
2. FCL (Framework Class Library): Provides reusable classes and methods.
3. JIT (Just-In-Time Compiler): Converts Intermediate Language (IL) into machine code.
4. Garbage Collector: Automatically manages unused memory.

Execution flow:

C# Code → C# Compiler → Intermediate Language (IL) → CLR → JIT Compiler → Machine Code

3. Basic Structure of a C# Program

Example:

```C#
using System;

namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

Explanation:

- using System: Imports the System namespace.
- namespace: Organizes related classes.
- class Program: Defines a class.
- static: Allows the method to be called without creating an object.
- void: Indicates that the method does not return a value.
- Main(): Entry point of the application.
- Console.WriteLine(): Displays output.

4. Variables and Constants

A variable is a named memory location used to store data.

Syntax:

```C#
dataType variableName = value;
```

Example:

```C#
int age = 21;
string name = "Devadharshini";
double salary = 25000.50;
```

A constant is a value that cannot be changed after declaration.

```C#
const double PI = 3.14;
```

5. Data Types in C#

Data types specify the type of value a variable can store.

Value Types:

- int – Integer values
- long – Large integer values
- float – Decimal values
- double – Double-precision decimal values
- decimal – High-precision decimal values
- char – Single character
- bool – True or false

Reference Types:

- string
- object
- class
- array
- interface
- delegate

Example:

```C#
int age = 21;
double mark = 89.5;
char grade = 'A';
bool isPassed = true;
```

```
string name = "Devadharshini";
```

6. Type Conversion

Type conversion means converting one data type into another.

Implicit Conversion:

Automatically performed when there is no data loss.

```
C#
int number = 10;
double result = number;
```

Explicit Conversion:

Requires manual casting.

```
C#
double number = 10.5;
int result = (int)number;
```

Convert Methods:

```
C#
string value = "100";
int number = Convert.ToInt32(value);
```

Parse Method:

```
C#
string value = "200";
int number = int.Parse(value);
```

TryParse Method:

```C#
string value = "300";

if (int.TryParse(value, out int number))
{
    Console.WriteLine(number);
}
```

TryParse is useful because it avoids an exception when the input is invalid.

7. Operators in C#

Operators are symbols used to perform operations on variables and values.

Arithmetic Operators:

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulus

Relational Operators:

- == Equal to
- != Not equal to
- > Greater than
- < Less than
- >= Greater than or equal to
- <= Less than or equal to

Logical Operators:

- && Logical AND
- || Logical OR
- ! Logical NOT

Assignment Operators:

- = Assignment
- += Add and assign
- -= Subtract and assign
- *= Multiply and assign
- /= Divide and assign

Increment and Decrement:

- ++ Increment
- -- Decrement

Example:

```
C#
int a = 10;
int b = 5;

Console.WriteLine(a + b);
Console.WriteLine(a > b);
Console.WriteLine(a == b);
```

8. Conditional Statements

Conditional statements execute code based on a condition.

if Statement:

```
C#
int age = 20;

if (age >= 18)
{
    Console.WriteLine("Eligible to vote");
}
```

if-else Statement:

```
C#
int number = 10;

if (number % 2 == 0)
{
    Console.WriteLine("Even");
}
else
```

```
{
    Console.WriteLine("Odd");
}
```

else-if Ladder:

```
C#
int mark = 85;

if (mark >= 90)
{
    Console.WriteLine("A Grade");
}
else if (mark >= 75)
{
    Console.WriteLine("B Grade");
}
else
{
    Console.WriteLine("C Grade");
}
```

switch Statement:

```
C#
int day = 2;

switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;

    case 2:
        Console.WriteLine("Tuesday");
        break;
```

```
    default:
        Console.WriteLine("Invalid Day");
        break;
}
```

9. Looping Statements

Loops are used to execute a block of code repeatedly.

for Loop:

```C#
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

while Loop:

```C#
int i = 1;

while (i <= 5)
{
    Console.WriteLine(i);
    i++;
}
```

do-while Loop:

```C#
int i = 1;

do
{
```

```
    Console.WriteLine(i);
    i++;
}
while (i <= 5);
```

foreach Loop:

```
C#
int[] numbers = { 10, 20, 30 };

foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

Difference:

- for: Used when the number of iterations is known.
- while: Checks the condition before execution.
- do-while: Executes at least once.
- foreach: Used to iterate through collections.

10. Jump Statements

Jump statements change the normal flow of execution.

- break: Terminates a loop or switch statement.
- continue: Skips the current iteration.
- return: Exits a method and optionally returns a value.

Example:

```
C#
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
    {
        continue;
```

```
    }

    Console.WriteLine(i);
}
```

Output: 1, 2, 4, 5

11. Methods in C#

A method is a block of code that performs a specific task.

Syntax:

```
C#
returnType MethodName(parameters)
{
    // Statements
}
```

Method without return value:

```
C#
void Display()
{
    Console.WriteLine("Welcome");
}
```

Method with return value:

```
C#
int Add(int a, int b)
{
    return a + b;
}
```

Calling a method:

```C#
int result = Add(10, 20);
Console.WriteLine(result);
```

Method Overloading:

Defining multiple methods with the same name but different parameter lists.

```C#
int Add(int a, int b)
{
    return a + b;
}

int Add(int a, int b, int c)
{
    return a + b + c;
}
```

12. Parameters and Arguments

A parameter is a variable declared in a method definition.

An argument is the actual value passed when calling the method.

Example:

```C#
void Display(string name)
{
    Console.WriteLine(name);
}

Display("Divya");
```

Here:

- name is the parameter.
- "Devadharshini" is the argument.

Parameter Passing:

1. Value: Passes a copy of the value.
2. ref: Passes a variable by reference; it must be initialized before passing.
3. out: Passes a variable by reference; the method must assign a value before returning.
4. params: Allows a variable number of arguments.

Example:

```C#
void Update(ref int number)
{
    number = 100;
}

int value = 10;
Update(ref value);
```

13. Strings in C#

A string is a sequence of characters.

Example:

```C#
string name = "CSharp";
```

Important string methods:

```C#
string text = "Hello World";

Console.WriteLine(text.Length);
Console.WriteLine(text.ToUpper());
Console.WriteLine(text.ToLower());
Console.WriteLine(text.Contains("Hello"));
Console.WriteLine(text.StartsWith("Hello"));
Console.WriteLine(text.EndsWith("World"));
Console.WriteLine(text.Replace("World", "CSharp"));
```

```
Console.WriteLine(text.Substring(0, 5));
```

String Interpolation:

```
C#
string name = "Devadharshini";
int age = 21;

Console.WriteLine($"My name is {name} and my age is {age}");
```

String is Immutable:

Once a string is created, its contents cannot be modified. Operations that appear to modify a string create a new string.

StringBuilder:

StringBuilder is used when frequent string modifications are required.

```
C#
using System.Text;

StringBuilder sb = new StringBuilder("Hello");
sb.Append(" World");

Console.WriteLine(sb);
```

14. Arrays

An array stores multiple values of the same data type in a fixed-size collection.

Declaration:

```
C#
int[] numbers = { 10, 20, 30, 40 };
```

Accessing elements:

```C#
Console.WriteLine(numbers[0]);
```

Array properties and methods:

```C#
Console.WriteLine(numbers.Length);
Array.Sort(numbers);
Array.Reverse(numbers);
```

Multidimensional Array:

```C#
int[,] matrix =
{
    { 1, 2 },
    { 3, 4 }
};

Console.WriteLine(matrix[0, 1]);
```

15. Collections in C#

Collections are used to store and manage groups of objects.

List:

A List is a dynamic collection that can grow or shrink.

```C#
List<int> numbers = new List<int>();

numbers.Add(10);
numbers.Add(20);
numbers.Add(30);

numbers.Remove(20);
```

```
foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

Dictionary:

Stores data as key-value pairs.

```
C#
Dictionary<int, string> students = new Dictionary<int, string>();

students.Add(1, "Arun");
students.Add(2, "Divya");

Console.WriteLine(students[1]);
```

Queue:

Follows FIFO (First In, First Out).

```
C#
Queue<int> queue = new Queue<int>();

queue.Enqueue(10);
queue.Enqueue(20);

Console.WriteLine(queue.Dequeue());
```

Stack:

Follows LIFO (Last In, First Out).

```
C#
Stack<int> stack = new Stack<int>();
```

```
stack.Push(10);
stack.Push(20);

Console.WriteLine(stack.Pop());
```

16. Object-Oriented Programming (OOP)

OOP is a programming approach based on classes and objects.

The four main principles are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Class:

A class is a blueprint for creating objects.

Object:

An object is an instance of a class.

Example:

```
C#
class Student
{
    public string Name;
    public int Age;

    public void Display()
    {
        Console.WriteLine(Name);
        Console.WriteLine(Age);
    }
}
```

```
class Program
{
    static void Main()
    {
        Student student = new Student();

        student.Name = "Devadharhini";
        student.Age = 21;

        student.Display();
    }
}
```

17. Encapsulation

Encapsulation means wrapping data and methods into a single unit and controlling access to the data.

It is implemented using access modifiers and properties.

Example:

```
C#
class Employee
{
    private int salary;

    public int Salary
    {
        get { return salary; }
        set { salary = value; }
    }
}
```

Advantages:

- Protects data.

- Provides controlled access.
- Improves maintainability.
- Supports data validation.

18. Access Modifiers

Access modifiers control the visibility of classes and members.

- public: Accessible from anywhere.
- private: Accessible only within the same class.
- protected: Accessible within the same class and derived classes.
- internal: Accessible within the same assembly.
- protected internal: Accessible within the same assembly or derived classes.
- private protected: Accessible within the same class or derived classes in the same assembly.

19. Constructors

A constructor is a special method that is automatically called when an object is created.

Characteristics:

- Same name as the class.
- No return type.
- Used to initialize objects.

Default Constructor:

```
C#
class Student
{
    public Student()
    {
        Console.WriteLine("Constructor Called");
    }
}
```

Parameterized Constructor:

```
C#
class Student
```

```
{
    public string Name;

    public Student(string name)
    {
        Name = name;
    }
}
```

Constructor Overloading:

A class can contain multiple constructors with different parameter lists.

20. Properties

Properties provide controlled access to private fields.

Example:

```C#
class Student
{
    private string name;

    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}
```

Auto-Implemented Property:

```C#
public string Name { get; set; }
```

Read-Only Property:

```C#
public string Name { get; }
```

21. Inheritance

Inheritance allows one class to acquire the properties and methods of another class.

The existing class is called the base class.

The new class is called the derived class.

Example:

```C#
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal is eating");
    }
}

class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog is barking");
    }
}
```

Usage:

```C#
Dog dog = new Dog();

dog.Eat();
dog.Bark();
```

Types of inheritance:

- Single inheritance
- Multilevel inheritance
- Hierarchical inheritance

C# does not support multiple inheritance through classes. Multiple interfaces can be implemented by a class.

22. Polymorphism

Polymorphism means one name or interface can represent different behaviors.

Two types:

Compile-Time Polymorphism:

Achieved through method overloading.

```
C#
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

Run-Time Polymorphism:

Achieved through method overriding using virtual and override keywords.

```
C#
class Animal
{
    public virtual void Sound()
```

```
    {
        Console.WriteLine("Animal sound");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

23. Abstraction

Abstraction means hiding implementation details and showing only essential functionality.

It can be achieved using abstract classes and interfaces.

Abstract Class:

```
C#
abstract class Animal
{
    public abstract void Sound();

    public void Eat()
    {
        Console.WriteLine("Eating");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Barking");
```

```
    }
}
```

Important points:

- An abstract class cannot be instantiated directly.
- It can contain abstract and non-abstract methods.
- A derived class must implement inherited abstract members unless it is also abstract.

24. Interfaces

An interface defines a contract that implementing classes must follow.

Example:

```C#
interface IPayment
{
    void Pay();
}

class UpiPayment : IPayment
{
    public void Pay()
    {
        Console.WriteLine("Payment using UPI");
    }
}
```

Advantages:

- Supports abstraction.
- Supports multiple interface implementation.
- Promotes loose coupling.
- Helps achieve flexible application design.

25. Static Keyword

The static keyword indicates that a member belongs to the class rather than an individual object.

Example:

```
C#
class Employee
{
    public static string Company = "ABC";
}
```

Accessing:

```
C#
Console.WriteLine(Employee.Company);
```

Important points:

- Static members can be accessed using the class name.
- An object is not required to access static members.
- Static classes cannot be instantiated.
- Static classes can contain only static members.

26. Struct, Enum and Nullable Types

Struct:

A struct is a value type used to represent small data structures.

```
C#
struct Point
{
    public int X;
    public int Y;
}
```

Enum:

An enum is a value type used to define named constants.

```
C#
enum Days
{
    Monday,
    Tuesday,
    Wednesday
}
```

Nullable Types:

Nullable value types can store null.

```
C#
int? age = null;
```

27. Exception Handling

An exception is a runtime error that interrupts normal program execution.

C# provides exception handling using:

- try
- catch
- finally
- throw

Example:

```
C#
try
{
    int a = 10;
    int b = 0;

    int result = a / b;
}
```

```
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Execution Completed");
}
```

Explanation:

- try: Contains code that may cause an exception.
- catch: Handles the exception.
- finally: Executes whether an exception occurs or not, subject to process termination.
- throw: Used to throw an exception.

28. Namespaces

A namespace is used to organize related classes and avoid naming conflicts.

Example:

```
C#
namespace MyApplication.Models
{
    class Student
    {
        public string Name;
    }
}
```

Using a namespace:

```
C#
using MyApplication.Models;
```

29. Delegates

A delegate is a type-safe reference to a method.

Example:

```C#
delegate void MyDelegate();

class Program
{
    static void Display()
    {
        Console.WriteLine("Hello");
    }

    static void Main()
    {
        MyDelegate del = Display;
        del();
    }
}
```

Uses:

- Callback methods.
- Event handling.
- Passing methods as parameters.

30. Events

Events are used to notify other parts of an application when something happens.

They are commonly implemented using delegates.

Example:

```C#
class Publisher
{
```

```
    public event Action? Notify;

    public void SendNotification()
    {
        Notify?.Invoke();
    }
}
```

An event subscriber can register a method to receive the notification.

31. Lambda Expressions

A lambda expression is a short way to write an anonymous function.

Syntax:

```
C#
(parameters) => expression
```

Example:

```
C#
Func<int, int, int> add = (a, b) => a + b;

Console.WriteLine(add(10, 20));
```

32. LINQ

LINQ stands for Language Integrated Query.

It is used to query and manipulate collections and other supported data sources.

Example:

```
C#
using System.Linq;
```

```
int[] numbers = { 10, 20, 30, 40, 50 };

var result = numbers.Where(n => n > 20);

foreach (int number in result)
{
    Console.WriteLine(number);
}
```

Important LINQ methods:

- Where(): Filters elements.
- Select(): Projects elements.
- OrderBy(): Sorts in ascending order.
- OrderByDescending(): Sorts in descending order.
- First(): Returns the first element.
- FirstOrDefault(): Returns the first element or default value.
- Single(): Returns exactly one matching element.
- Any(): Checks whether any element matches.
- All(): Checks whether all elements match.
- Count(): Returns the number of elements.
- Sum(): Calculates the sum.
- Average(): Calculates the average.

33. File Handling

File handling allows applications to create, read, write and delete files.

Example:

```
C#
using System.IO;

File.WriteAllText("sample.txt", "Hello CSharp");

string content = File.ReadAllText("sample.txt");

Console.WriteLine(content);
```

Important methods:

- File.Create()
- File.ReadAllText()
- File.WriteAllText()
- File.AppendAllText()
- File.Delete()
- File.Exists()

34. Asynchronous Programming

Asynchronous programming allows operations to execute without blocking the calling thread while waiting for an asynchronous operation.

Important keywords:

- async
- await
- Task
- Task<T>

Example:

```
C#
async Task DisplayAsync()
{
    await Task.Delay(2000);

    Console.WriteLine("Completed");
}
```

Task:

Represents an asynchronous operation that does not return a value.

Task<T>:

Represents an asynchronous operation that returns a value of type T.

Example:

```C#
async Task<int> GetNumberAsync()
{
    await Task.Delay(1000);

    return 100;
}
```

35. C# Concepts Used in ASP.NET Core

Dependency Injection (DI):

Dependency Injection is a design pattern used to provide required objects to a class instead of creating them manually.

Example:

```C#
public class StudentService
{
    private readonly ILogger<StudentService> _logger;

    public StudentService(ILogger<StudentService> logger)
    {
        _logger = logger;
    }
}
```

Service Lifetimes:

1. Transient: Creates a new instance every time it is requested.
2. Scoped: Creates one instance per request scope.
3. Singleton: Creates one instance for the application's lifetime.

Entity Framework Core:

EF Core is an Object-Relational Mapper (ORM) that allows developers to work with databases using C# objects.

1. DbContext

Definition

`DbContext` is a class provided by Entity Framework Core. It acts as a bridge between your C# application and the database.

It manages:

- Database connections.
- Database queries.
- Tracking changes made to objects.
- Saving changes to the database.
- Managing entities and their relationships.

Simple understanding:

Think of `DbContext` as a manager that communicates with the database on behalf of your application.

Real-world example

Suppose you are developing a Student Management System.

You need to:

- Add a student.
- Display all students.
- Update student details.
- Delete a student.

Instead of writing SQL queries manually for every operation, you use `DbContext` to perform these operations using C# objects.

How to create DbContext?

```C#
using Microsoft.EntityFrameworkCore;


public class AppDbContext : DbContext
```

```
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

Let's understand this code:

`AppDbContext : DbContext`

- `AppDbContext` is our own context class.
- `:` represents inheritance in C#.
- `DbContext` is the EF Core base class.
- By inheriting DbContext, our class gets EF Core database functionality.

`DbContextOptions<AppDbContext> options`

- Receives the configuration required by AppDbContext.
- Includes information such as which database provider to use and the connection string.

`: base(options)`

- Calls the constructor of the parent class, DbContext.
- Passes the configuration to the EF Core base class.

`DbSet<Student> Students`

- Represents the collection of Student entities that we work with through this context.
- We use it to query and perform operations on student records.

2. DbSet

Definition

`DbSet<TEntity>` represents a collection of entities of a particular type that EF Core can query and manage.

In simple words, DbSet is used to perform database operations on a particular entity or table.

Example

```
C#
public DbSet<Student> Students { get; set; }
```

Here:

- `DbSet` is an EF Core class.
- `Student` is the entity type.
- `Students` is the property name.

Suppose your Student class is:

```
C#
public class Student

{

    public int Id { get; set; }


    public string Name { get; set; }


    public int Age { get; set; }

}
```

When EF Core maps this entity to a database table, the `Students` DbSet represents the student records you can work with.

What operations can we perform using DbSet?

1. Insert a record

```
C#
Student student = new Student

{

    Name = "Divya",

    Age = 21

};



_context.Students.Add(student);



await _context.SaveChangesAsync();
```

Explanation:

- `Add()` marks the student as a new entity.
- `SaveChangesAsync()` saves the pending changes to the database.

2. Retrieve records

```
C#
var students = await _context.Students.ToListAsync();
```

This retrieves all student records.

3. Retrieve a specific record

```
C#
var student = await _context.Students

    .FirstOrDefaultAsync(s => s.Id == 1);
```

This retrieves the student whose Id is 1, or null if no matching student exists.

4. Update a record

```C#
var student = await _context.Students
    .FirstOrDefaultAsync(s => s.Id == 1);

if (student != null)
{
    student.Name = "Arun";

    await _context.SaveChangesAsync();
}
```

5. Delete a record

```C#
var student = await _context.Students
    .FirstOrDefaultAsync(s => s.Id == 1);

if (student != null)
{
    _context.Students.Remove(student);

    await _context.SaveChangesAsync();
```

```
}
```

Important point

DbSet provides access to entity operations, but changes are normally persisted when you call:

```
C#
await _context.SaveChangesAsync();
```

3. DbContextOptions

Definition

`DbContextOptions` contains the configuration required by DbContext to connect to and work with a database.

It specifies things such as:

- Which database provider to use.
- Database connection string.
- Other EF Core configuration settings.

Simple understanding:

DbContext is the manager.

DbSet represents the entity collection.

DbContextOptions contains the instructions needed to configure the manager.

How to configure DbContextOptions?

In ASP.NET Core, we normally configure it in `Program.cs`.

```
C#
using Microsoft.EntityFrameworkCore;


builder.Services.AddDbContext<AppDbContext>(options =>
```

```
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")

    )

);
```

Let's understand this code:

`AddDbContext<AppDbContext>()`

- Registers AppDbContext with the Dependency Injection container.
- Allows ASP.NET Core to provide the context wherever it is required.

`options`

- Represents a `DbContextOptionsBuilder<AppDbContext>` object.
- It is used to configure the context.

`UseSqlServer()`

- Specifies SQL Server as the database provider.

`GetConnectionString("DefaultConnection")`

- Reads the connection string named `DefaultConnection` from the application configuration.

Connection string in appsettings.json

```
C#

{

  "ConnectionStrings": {

    "DefaultConnection":
"Server=.;Database=StudentDB;Trusted_Connection=True;TrustServerCer
tificate=True;"

  }
```

```
}
```

This tells EF Core which SQL Server database to connect to.